



XDBC Gateway

Version 2026.1
2026-04-20

XDBC Gateway

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 XDBC Gateway Overview	1
2 Connecting with the XDBC Gateway	3
2.1 Starting the External Language Server	3
2.2 Establishing the Connection	3
2.2.1 SQL Gateway Connection Definition	4
2.2.2 Connection String	4
3 Using the XDBC Gateway	7
3.1 Statements	7
3.2 Prepared Statements	8
3.3 Stored Procedures	9
3.4 Result Sets	11

1

XDBC Gateway Overview

The XDBC Gateway provides a single interface for connecting InterSystems IRIS® to an external database with JDBC or ODBC. It acts as the modern, streamlined successor to the [SQL Gateway](#).

The XDBC Gateway acts like a JDBC or ODBC driver and lets you perform any of the following operations:

- Access data stored in third-party relational databases within InterSystems IRIS applications.
- Store persistent InterSystems IRIS objects in external relational databases.
- Create class methods that perform the same actions as corresponding external stored procedures.

See the following topics for more information:

- [Connecting with the XDBC Gateway](#) describes how to establish a connection between InterSystems IRIS and an external database with XDBC.
- [Using the XDBC Gateway](#) describes how to execute statements, create stored procedures, parameterize queries, and view query results.

2

Connecting with the XDBC Gateway

At a high level, an XDBC Gateway connection between InterSystems IRIS and an external database consists of the following:

1. [Starting the External Language Server](#) for your connection type (JDBC or ODBC).
2. [Connecting](#) with `%XDBC.Gateway.Connection:GetConnection()`, passing in the connection information for the external database. This information can be either a named SQL Gateway connection definition (which requires a DSN) or a JSON connection string.

After you establish a connection to the external database, you can use the methods provided by the `%XDBC` package to interact with it. For details, see [Using the XDBC Gateway](#).

2.1 Starting the External Language Server

InterSystems IRIS provides predefined External Language Servers for JDBC and ODBC connections through the XDBC Gateway: **%Java Server** and **%ODBC Server**. These are stopped by default and start automatically when a client attempts to connect, but starting them manually can be helpful for troubleshooting connection issues:

1. Go to **System > Configuration > External Language Servers**.
2. Click **Start** on the server for your connection type:
 - JDBC: **%Java Server**
 - ODBC: **%ODBC Server**

2.2 Establishing the Connection

After [starting the External Language Server](#), you can establish a connection between InterSystems IRIS and the external database by passing the connection information to the `GetConnection()` method. `GetConnection` takes as an argument the connection information of the external database, which can be any of the following:

- The name of a [SQL Gateway connection definition](#)
- JSON (either a dynamic object or JSON string) containing the connection information to the external database.

2.2.1 SQL Gateway Connection Definition

To connect with a named SQL Gateway connection definition:

1. Create a SQL Gateway Connection definition for your connection type:
 - [JDBC](#)
 - [ODBC](#)
2. Specify the definition name in %XDBC.Gateway.Connection.**GetConnection()**:

```
set conn = ##class(%XDBC.Gateway.Connection).GetConnection("ConnectionDefinitionName",.status)
```

2.2.2 Connection String

To connect with a connection string, create a JSON string or dynamic object and pass it to **GetConnection()**. The fields vary between connection types.

2.2.2.1 JDBC

JDBC connection strings all require a host, port, database name, username, password, connection type, and the information for the JDBC driver. When the connection is to a first-party database (that is, another instance of InterSystems IRIS), you can set the JDBC driver by specifying `com.intersystems.jdbc.IRISDataSource`.

A connection to another InterSystems IRIS database uses the following fields:

```
{
  "host": "external_db_hostname",
  "port": external_db_port,
  "dbname": "external_db_name",
  "datasource": "com.intersystems.jdbc.IRISDataSource",
  "user": "external_db_user",
  "password": "external_db_password",
  "type": "jdbc"
}
```

When the connection is to a third-party database, you can set the JDBC driver by specifying the location of the driver's .jar file with `classpath` and the JDBC DataSource with `datasource`. A connection to a Microsoft SQL Server database uses the following fields:

```
{
  "host": "external_db_hostname",
  "port": external_db_port,
  "dbname": "external_db_name",
  "datasource": "com.microsoft.sqlserver.jdbc.SQLServerDataSource",
  "classpath": "/path/to/sqljdbc42.jar",
  "properties": "null",
  "user": "external_db_user",
  "password": "external_db_password",
  "type": "jdbc"
}
```

The following examples connect to an InterSystems IRIS and Microsoft SQL Server database with JDBC using the above fields.

Connect to InterSystems IRIS:

```
set connInfo =
{"host":"192.0.2.0","port":1972,"dbname":"USER","datasource":"com.intersystems.jdbc.IRISDataSource","user":"SYS","password":"SYS","type":"jdbc"}
set conn = ##class(%XDBC.Gateway.Connection).GetConnection(connInfo,.status)
```


Connect to Microsoft SQL Server:

```
set connInfo =
{"type": "ODBC", "dsn": "my_dsn", "username": "sa", "password": "my_password", "type": "odbc"}
set conn = ##class(%XDBC.Gateway.Connection).GetConnection(connInfo, .status)
```

2.2.2.2 ODBC

ODBC connection strings can either be a DSN with a username and password or a string with the connection information that would normally be provided by the DSN.

To connect with a DSN, create a DSN in your system and specify it with the dsn field:

```
{
  "dsn": "dsn_name",
  "user": "external_db_user",
  "password": "external_db_password",
  "type": "odbc"
}
```

To connect without a DSN, you must specify in the string field the connection information that would normally be provided by the DSN. Different databases expect different things, so the contents of this string vary between databases:

- DSN-less connection to InterSystems IRIS:

```
{
  "string" : "Driver=InterSystems IRIS
ODBC35;HOST=192.0.2.0;Port=1972;Database=USER;UID=_SYSTEM;PWD=SYS;LOG=1"
  "type" : "odbc"
}
```

- DSN-less connection to Microsoft SQL Server:

```
{
  "string" : "Driver=SQL Server;Server=MSSQLServer;Database=XDBCtest;UID=sa;PWD=my_password"
  "type" : "odbc"
}
```

The following examples connect to an InterSystems IRIS and a Microsoft SQL Server database with ODBC and both with and without a DSN.

Connect with a DSN to any external database:

```
s connInfo = {"type": "odbc", "dsn": "my_dsn", "username": "_SYSTEM", "password": "SYS"}
set conn = ##class(%XDBC.Gateway.Connection).GetConnection(connInfo, .status)
```

Connect without a DSN to InterSystems IRIS:

```
s connInfo= {"type": "odbc", "string": "Driver=InterSystems IRIS
ODBC35;HOST=192.0.2.0;Port=1972;Database=USER;UID=_SYSTEM;PWD=SYS;LOG=1"}
set conn = ##class(%XDBC.Gateway.Connection).GetConnection(connInfo, .status)
```

Connect without a DSN to Microsoft SQL Server:

```
s connInfo= {"type": "odbc", "string": "Driver=SQL
Server;Server=MSSQLServer;Database=XDBCtest;UID=sa;PWD=my_password"}
set conn = ##class(%XDBC.Gateway.Connection).GetConnection(connInfo, .status)
```


3

Using the XDBC Gateway

The following section gives an overview of the basic usage of the various SQL APIs provided by the %XDBC package. In particular, this page covers:

- Statements
- Prepared statements
- Stored procedures
- Result sets

The examples in this page build on the example table created in the [Statements section](#) and assume that you already know how to [configure a connection between InterSystems IRIS and your external database with the XDBC Gateway](#).

Note: Some functionality provided by the %XDBC package and its methods might be limited or unavailable depending on the features of the external database. For example, Microsoft SQL Server [does not support `setSchema\(\)`](#) and InterSystems IRIS only supports [two transaction isolation levels](#). For details, refer to your external database's documentation.

3.1 Statements

To create a statement, use **CreateStatement()**:

```
set conn = ##class(%XDBC.Gateway.Connection).GetConnection("ExampleJDBC")
set statement = conn.CreateStatement()
```

To **Execute** a statement, use one of the following depending on the statement type:

- %XDBC.Gateway.Statement.**ExecuteUpdate()**: Executes a DDL or non-**SELECT** DML statement and returns the number of rows affected.
- %XDBC.Gateway.Statement.**ExecuteQuery()**: Executes a **SELECT** statement and returns the results as an instance of %XDBC.Gateway.ResultSet.
- %XDBC.Gateway.Statement.**ExecuteBatch()**: Executes a batch of statements and, if all statements succeed, returns the number of rows affected as an array. You can add statements with **AddBatch()**.

AddBatch() and **ExecuteBatch()** are only supported by JDBC. For the equivalents in ODBC use [prepared statements](#) with **AddBatchEx()** and **ExecuteBatchEx()**.

- **%XDBC.Gateway.Statement.Execute()**: Executes any statement and returns a boolean if results were from the statement. A result in this context is either a **ResultSet** (retrieved with **getResultSet()**) or the number of rows updated (retrieved with **getUpdateCount()**). If **Execute()** returns multiple results, you can retrieve them with **getMoreResults()** (after retrieving the first result with either **getResultSet()** or **getUpdateCount()**). In general, you should only use this for statements that can return multiple types of results.

The following example uses the various statement execution methods to create and insert values into a table:

1. **CREATE** a table:

```
set res = statement.ExecuteUpdate("CREATE TABLE Contacts (first_name VARCHAR(50), last_name VARCHAR(50), phone VARCHAR(20))")
```

2. **INSERT** data into the table. For demonstration purposes, this example uses both individual and batch statements:

```
do statement.ExecuteUpdate("INSERT INTO Contacts VALUES ('John', 'Doe', '892-555-3819')")
do statement.AddBatch("INSERT INTO Contacts VALUES ('Jan', 'Kowalski', '563-555-0662')")
do statement.AddBatch("INSERT INTO Contacts VALUES ('Anna', 'Kowalska', '779-555-9984')")
do statement.ExecuteBatch()
```

3. Query the table:

```
set res = statement.ExecuteQuery("SELECT * FROM Contacts")
do res.%Display()
```

Output:

first_name	last_name	phone
John	Doe	892-555-3819
Jan	Kowalski	563-555-0662
Anna	Kowalska	779-555-9984

3 Rows(s) Affected

3.2 Prepared Statements

You can parameterize statements with prepared statements. To create a prepared statement, use **XDBC.Gateway.Connection.PrepareStatement()** and use the question mark (?) character for each parameter:

```
set conn = ##class(%XDBC.Gateway.Connection).GetConnection("ExampleJDBC")
set prepared = conn.PrepareStatement("INSERT INTO Contacts(first_name, last_name, phone) VALUES(?,?,?)")
```

Parameters are 1-indexed in left-to-right ascending order. To set the parameter, use the **Set()** method that corresponds to its type (these vary between JDBC and ODBC). The above column types are all VARCHARs, so set them with **SetString()**:

```
do prepared.SetString(1, "Jane")
do prepared.SetString(2, "Doe")
do prepared.SetString(3, "391-555-1883")
```

You can then execute the prepared statement with an **Execute** method, depending on the statement type:

- **ExecuteUpdate()**: Executes a prepared DDL statement (such as **INSERT**, **UPDATE**, and **DELETE**) and returns the number of rows affected.
- **ExecuteQuery()**: Executes a prepared **SELECT** statement and returns the results as an instance of **%XDBC.Gateway.ResultSet**.
- **ExecuteBatch()**: Executes a batch of prepared statements and, if all statements succeed, returns the number of rows affected as an array. You can add statements with **AddBatch()**.

- **Execute()**: Executes any prepared statement and returns a boolean if results were returned by the statement. A result in this context is either a **ResultSet** (retrieved with **getResultSet()**) or the number of rows updated (retrieved with **getUpdateCount()**). If **Execute()** returns multiple results, you can retrieve them with **getMoreResults()** (after retrieving the first result with either **getResultSet()** or **getUpdateCount()**). In general, you should only use this for statements that can return multiple types of results.

The above example is an **INSERT** statement, so you can execute it with **ExecuteUpdate()**:

```
do prepared.ExecuteUpdate()
```

3.3 Stored Procedures

Support for stored procedures differs slightly between JDBC and ODBC.

In JDBC, you can use an existing stored procedure to create an instance of **CallableStatement** and then call it with **Execute()**:

1. **Connect** to the external database with JDBC:

```
set conn = ##class(%XDBC.Gateway.Connection).GetConnection("ExampleJDBC")
```

2. Create the stored procedure:

```
do statement.ExecuteUpdate("CREATE PROCEDURE addCountryCode(IN prefix VARCHAR(20)) BEGIN UPDATE
Contacts SET phone = {fn CONCAT(:prefix, phone)} WHERE phone NOT LIKE {fn CONCAT(:prefix, '+%')};
SELECT * FROM Contacts; END")
```

3. Create an instance of **CallableStatement** with **PrepareCall()**, specifying a **CALL** to the stored procedure and using ? (question marks) as placeholders for each argument:

```
set callable = conn.PrepareCall("CALL addCountryCode(?)")
```

4. Set the arguments for the procedure call with the **Set()** method that matches the type of your arguments. In this example, the argument is a **VARCHAR**, so use **SetString()**:

```
do callable.SetString(1, "+1")
```

For a full list of **Set()** methods, see **CallableStatement**.

5. Call the stored procedure with **Execute()**:

```
do callable.Execute()
```

Results:

```
set statement = conn.createStatement()
set res = statement.executeQuery("SELECT * FROM Contacts")
do res.%Display()
```

first_name	last_name	phone
John	Doe	+1892-555-3819
Jan	Kowalski	+1563-555-0662
Anna	Kowalska	+1779-555-9984

3 Rows(s) Affected

In ODBC, you can specify the **CALL** to the stored procedure directly with a **Statement** or parameterize it with **PreparedStatement**.

To use a **Statement**:

1. [Connect](#) to the external database with ODBC:

```
set conn = ##class(%XDBC.Gateway.Connection).GetConnection("ExampleODBC")
```

2. Create a Statement:

```
set statement = conn.CreateStatement()
```

3. Create the stored procedure:

```
do statement.ExecuteUpdate("CREATE PROCEDURE addCountryCode(IN prefix VARCHAR(20)) BEGIN UPDATE  
Contacts SET phone = {fn CONCAT(:prefix, phone)} WHERE phone NOT LIKE {fn CONCAT(:prefix, '%')};  
SELECT * FROM Contacts; END")
```

4. Call the stored procedure with the appropriate **Execute()** method and **CALL**. In this case, the stored procedure contains an **INSERT** statement, so use **ExecuteUpdate()**:

```
do statement.ExecuteUpdate("CALL addCountryCode('+1')")
```

Results:

```
set statement = conn.CreateStatement()  
set res = statement.ExecuteQuery("SELECT * FROM Contacts")  
do res.%Display()
```

first_name	last_name	phone
John	Doe	+1892-555-3819
Jan	Kowalski	+1563-555-0662
Anna	Kowalska	+1779-555-9984

3 Rows(s) Affected

To use a PreparedStatement:

1. [Connect](#) to the external database with ODBC:

```
set conn = ##class(%XDBC.Gateway.Connection).GetConnection("ExampleODBC")
```

2. Create the stored procedure:

```
do statement.ExecuteUpdate("CREATE PROCEDURE addCountryCode(IN prefix VARCHAR(20)) BEGIN UPDATE  
Contacts SET phone = {fn CONCAT(:prefix, phone)} WHERE phone NOT LIKE {fn CONCAT(:prefix, '%')};  
SELECT * FROM Contacts; END")
```

3. Create a PreparedStatement:

```
set prepared = conn.PrepareStatement("CALL addCountryCode(?)")
```

4. Set the arguments for the procedure call with the **Set()** method that matches the type of your arguments. In this example, the argument is a VARCHAR, so use **SetString()**:

```
do prepared.SetString(1, "+1")
```

For a full list of **Set()** methods, see PreparedStatement.

5. Call the stored procedure with the appropriate **Execute()** method. In this case, the stored procedure contains an **UPDATE** statement, so use **ExecuteUpdate()**:

```
do prepared.ExecuteUpdate()
```

Results:

```
set statement = conn.createStatement()
set res = statement.executeQuery("SELECT * FROM Contacts")
do res.%Display()

first_name    last_name    phone
John          Doe          +1892-555-3819
Jan           Kowalski     +1563-555-0662
Anna          Kowalska     +1779-555-9984

3 Rows(s) Affected
```

3.4 Result Sets

Query results are returned in subclasses of the `%XDBC.Gateway.ResultSet` interface, the particular implementation of which depends on your connection method (JDBC or ODBC). Result sets can be conceptualized as a set of rows and a cursor, which points to a row after a call to **Next()**.

To access a result set row-by-row, use **Next()** to verify that a next row exists (returns 1 if the next row exists and 0 otherwise) and advance the cursor to the next row, and use **Get()** to retrieve the data in the column specified by either a column index or name.

The following example queries the `Contacts` table which contains the data:

first_name	last_name	phone
John	Doe	892-555-3819
Jan	Kowalski	563-555-0662
Anna	Kowalska	779-555-9984

1. Query the table to get a `ResultSet`:

```
set res = statement.executeQuery("SELECT * FROM Contacts")
```

2. Use **Next()** to advance the cursor and verify that a valid row exists, then get the data from each column with **Get()**. This example passes both column indices and names to **Get()** for demonstration purposes:

```
while result.Next() {
    write !, "Name: ", res.Get(1), " ", res.Get("last_name"), !, "Phone: ", res.Get(3)
}
```

Output:

```
Name: John Doe
Phone: 892-555-3819

Name: Jan Kowalski
Phone: 1563-555-0662

Name: Anna Kowalska
Phone: 779-555-9984
```

To retrieve an entire row as a list, use **GetRow()**:

```
do res.Next()
zwrite res.GetRow()
$lb("John", "Doe", "892-555-3819")
```

To display all rows at once, use **%Display()**. This advances the cursor to the end of the result set:

```
do res.%Display()
```

first_name	last_name	phone
John	Doe	892-555-3819
Jan	Kowalski	563-555-0662
Anna	Kowalska	779-555-9984

```
3 Rows(s) Affected
```